

Optimizador Inteligente de Currículums con Multi-Agentes y RAG

Descripción del Proyecto

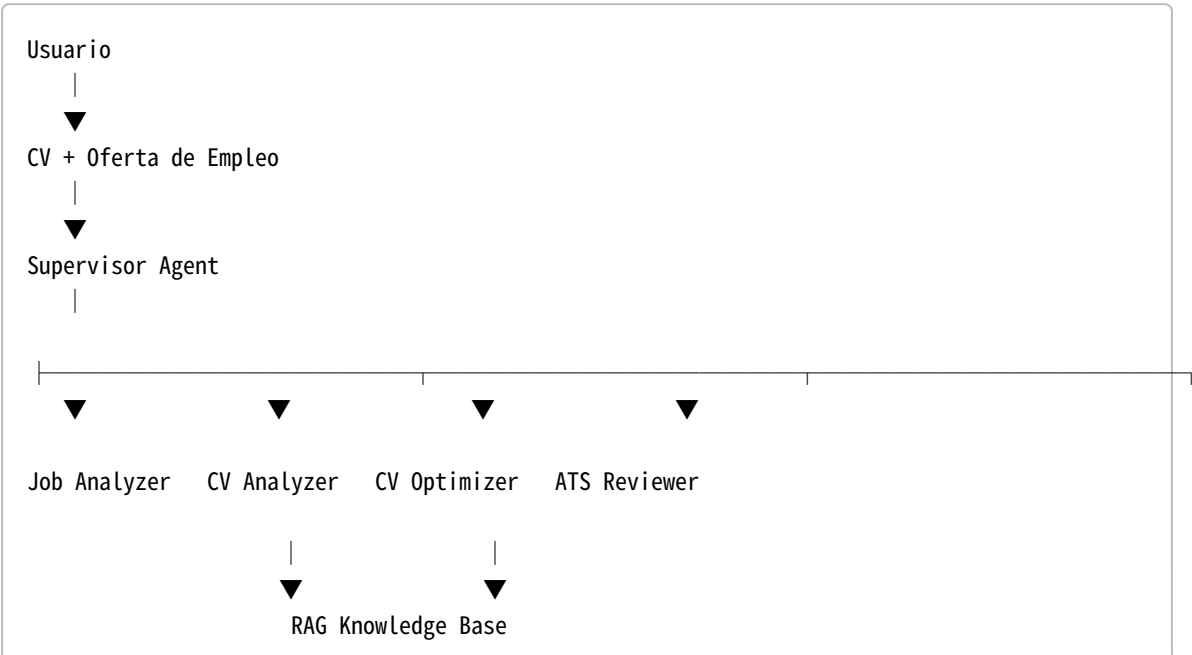
Este proyecto consiste en desarrollar un sistema inteligente basado en **Multi-Agentes**, **LangGraph**, **Ollama** y **RAG (Retrieval-Augmented Generation)** capaz de analizar una oferta de empleo, optimizar un currículum vitae para adaptarlo a dicha oferta, evaluar su compatibilidad con sistemas ATS (Applicant Tracking Systems) y generar una versión final mejorada.

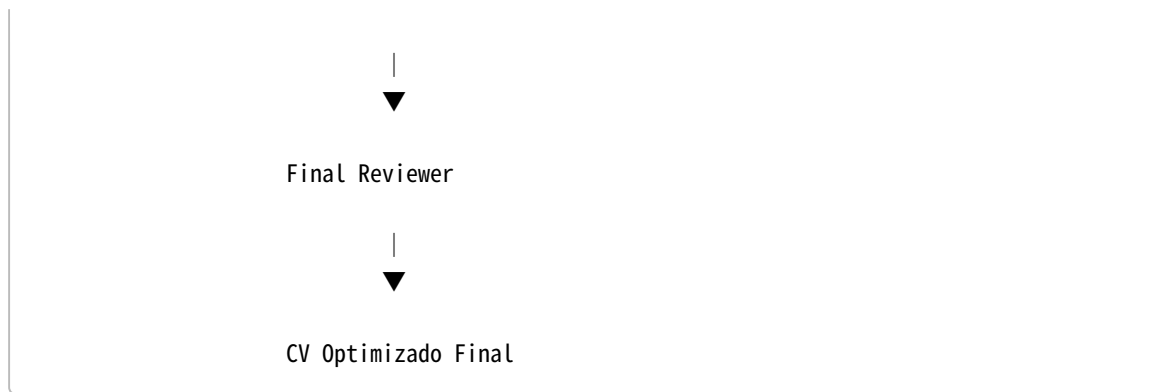
El objetivo es aumentar las posibilidades de que un candidato supere los filtros automáticos utilizados por las empresas durante los procesos de selección.

Objetivos

- Analizar automáticamente una oferta de empleo.
- Extraer requisitos y palabras clave relevantes.
- Analizar el contenido de un currículum.
- Adaptar el CV a una oferta específica.
- Evaluar la compatibilidad con sistemas ATS.
- Generar una versión optimizada y lista para enviar.
- Utilizar una arquitectura Multi-Agente coordinada mediante LangGraph.
- Integrar una base de conocimiento RAG para mejorar la calidad de las recomendaciones.

Arquitectura General





Tecnologías Utilizadas

- Python
- LangGraph
- LangChain
- Ollama
- Qwen3:8B
- ChromaDB
- nomic-embed-text
- PyPDFLoader

Agentes del Sistema

1. Job Analyzer Agent

Función

Analizar la oferta de empleo y extraer:

- Habilidades requeridas
- Tecnologías
- Nivel de experiencia
- Palabras clave relevantes

Entrada

Oferta de empleo.

Salida

```
{  
  "skills": [  
    "Python",  
    "Docker",
```

```
"AWS",  
"FastAPI"  
],  
"seniority": "Mid-Level"  
}
```

2. CV Analyzer Agent

Función

Analizar el currículum del candidato.

Información extraída

- Experiencia laboral
- Formación académica
- Certificaciones
- Competencias técnicas

Salida

```
{  
  "skills": [  
    "Python",  
    "Flask",  
    "SQL"  
  ]  
}
```

3. CV Optimizer Agent

Función

Comparar la oferta de empleo con el CV y generar una versión optimizada.

Tareas

- Reescribir descripciones profesionales.
- Destacar logros cuantificables.
- Incorporar palabras clave relevantes.
- Adaptar el perfil al puesto solicitado.

Integración RAG

Consulta documentación especializada sobre:

- Buenas prácticas para CV.
 - Ejemplos de currículums exitosos.
 - Recomendaciones de reclutadores.
-

4. ATS Reviewer Agent

Función

Simular el comportamiento de un sistema ATS.

Evaluación

- Coincidencia de palabras clave.
- Cobertura de requisitos.
- Compatibilidad ATS.

Ejemplo de salida

Compatibilidad ATS: 85%

Palabras clave encontradas:

- Python
- Docker
- Git

Palabras clave faltantes:

- AWS

Integración RAG

Consulta documentación relacionada con:

- ATS.
 - Procesos de selección automatizados.
 - Estrategias de optimización para CV.
-

5. Final Reviewer Agent

Función

Realizar una revisión final antes de entregar el resultado.

Responsabilidades

- Corrección gramatical.
- Mejora de la redacción.
- Eliminación de redundancias.
- Uniformidad de formato.

Resultado

CV final optimizado y listo para enviar.

Base de Conocimiento RAG

Documentos PDF

```
ats_rules.pdf  
  
resume_best_practices.pdf  
  
linkedin_resume_guide.pdf  
  
tech_resume_examples.pdf  
  
recruitment_tips.pdf
```

Flujo de Procesamiento RAG

```
PDF  
↓  
PyPDFLoader  
↓  
Chunking  
↓  
Embeddings (nomic-embed-text)  
↓  
ChromaDB  
↓  
Retrieval  
↓  
Agentes
```

Flujo Completo del Sistema

1. Usuario carga CV y oferta de empleo.
2. Job Analyzer extrae requisitos y habilidades.
3. CV Analyzer analiza el currículum.
4. CV Optimizer adapta el contenido al puesto.
5. ATS Reviewer calcula la compatibilidad ATS.
6. Final Reviewer realiza una revisión final.
7. Se genera el CV optimizado.

Estructura del Proyecto

```
proyecto/
├── docs/
│   ├── ats_rules.pdf
│   ├── resume_best_practices.pdf
│   ├── linkedin_resume_guide.pdf
│   ├── tech_resume_examples.pdf
│   └── recruitment_tips.pdf
├── chroma_db/
├── agents/
│   ├── job_analyzer.py
│   ├── cv_analyzer.py
│   ├── cv_optimizer.py
│   ├── ats_reviewer.py
│   └── final_reviewer.py
├── graph/
│   └── workflow.py
├── rag/
│   └── vector_store.py
├── main.py
└── README.md
```

Resultados Esperados

- Optimización automática de currículums.
 - Mayor compatibilidad con sistemas ATS.
 - Uso de arquitectura Multi-Agente.
 - Aplicación práctica de LangGraph.
 - Integración de RAG para mejorar la calidad de las recomendaciones.
 - Generación de documentos personalizados para cada oferta de empleo.
-

Posibles Mejoras Futuras

- Interfaz web con Streamlit.
- Exportación automática a PDF.
- Comparación entre varias ofertas de empleo.
- Integración con LinkedIn.
- Análisis de cartas de presentación.
- Traducción automática del CV a diferentes idiomas.